ASSIGNMENT-17.3

Name: E.Ramya

Ht.no: 2303A51282

Batch: 05

Task 1: Employee Dataset Cleaning
Task:
Use AI assistance to clean an employee information dataset.
Instructions:
• Handle missing values in salary, designation, and experience
• Remove duplicate employee IDs
• Standardize text values (job titles in lowercase)
• Encode categorical features (designation, employment_type)

Prompt:

Clean my employee dataset using Python. Fill missing values in salary, designation, and experience, remove duplicate employee IDs, convert designation to lowercase, encode designation and employment_type, and save the cleaned data as cleaned_employee_dataset.csv.

Clean my employee dataset using Python. Fill missing values in salary, designation, and experience, remove duplicate employee IDs, convert designation to lowercase, encode designation and employment_type, and save the cleaned data as cleaned_employee_dataset.csv. Explain the code in

Code:

```
# Fill missing 'salary' with the median
df['salary'].fillna(df['salary'].median(), inplace=True)

# Fill missing 'designation' with
df['designation'].fillna('Unknown', inplace=True)

# Fill missing 'experience' with the median
df['experience'].fillna(df['experience'].median(), inplace=True)

print("DataFrame after filling missing values:")
display(df.head())
```

```
# Remove duplicate 'employee_id' entries, keeping the first occurrence
df.drop_duplicates(subset=['employee_id'], inplace=True)

print("DataFrame after removing duplicate employee IDs:")
display(df.head())
```

```
# Convert 'designation' column to lowercase
df['designation'] = df['designation'].str.lower()

print("DataFrame after converting designation to lowercase:")
display(df.head())
```

```
# One-hot encode 'designation' and 'employment_type'
df = pd.get_dummies(df, columns=['designation', 'employment_type'], dtype=int)

print("DataFrame after one-hot encoding categorical features:")
display(df.head())
```

```
# Save the cleaned DataFrame to a new CSV file
df.to_csv('cleaned_employee_dataset.csv', index=False)

print("Cleaned data saved to 'cleaned_employee_dataset.csv'.")
```

Output:

```
DataFrame after filling missing values:
/tmp/ipykernel_5901/3061045955.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or S
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object o

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=Tru


  df['salary'].fillna(df['salary'].median(), inplace=True)
/tmp/ipykernel_5901/3061045955.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or S
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object o

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=Tru


  df['designation'].fillna('Unknown', inplace=True)
/tmp/ipykernel_5901/3061045955.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or S
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object o

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=Tru


  df['experience'].fillna(df['experience'].median(), inplace=True)
```

	employee_id	name	salary	designation	experience	employment_type
0	1	Alice	50000.0	Manager	2.0	Full-time
1	2	Bob	60000.0	Developer	5.0	Contract
2	3	Charlie	62000.0	Developer	3.0	Full-time
3	4	David	75000.0	Analyst	4.5	Full-time
4	5	Eve	80000.0	Manager	6.0	Part-time

DataFrame after removing duplicate employee IDs:

	employee_id	name	salary	designation	experience	employment_type
0	1	Alice	50000.0	Manager	2.0	Full-time
1	2	Bob	60000.0	Developer	5.0	Contract
2	3	Charlie	62000.0	Developer	3.0	Full-time
3	4	David	75000.0	Analyst	4.5	Full-time
4	5	Eve	80000.0	Manager	6.0	Part-time

Explanation:

- This prompt is used to clean the employee dataset using Python.
- It helps to fill missing values in salary, designation, and experience.
- It removes duplicate employee IDs to avoid repeated records.

- It converts designation values into lowercase for a standard format.
- It encodes categorical columns like designation and employment type into numbers.
- After cleaning, the dataset becomes ready for HR analytics and prediction tasks.

Task 2 – Student Performance Data Preprocessing
Task:
Preprocess student performance data using AI assistance.
Instructions:
• Convert exam_date column into datetime format
• Extract new features (month, semester, day)
• Normalize total_marks
• Detect and handle outliers in marks using IQR or Z-score

Prompt:

In this task, the student performance dataset is cleaned and prepared for analysis. First, the **exam_date** column is converted into **datetime format** so that the date can be processed correctly. Then, new features such as **month, semester, and day** are extracted from the exam date to get more useful information from the dataset. After that, the **total_marks** column is **normalized** so that all values are brought into a similar range, which helps improve model performance. Finally, **outliers** in marks are detected and handled using **IQR or Z-score methods** to reduce the effect of unusual values. This preprocessing makes the dataset more

accurate, clean, and suitable for **student performance analysis and prediction**.

In this task, the student performance dataset is cleaned and prepared for analysis. First, the exam_date column is converted into datetime format so that the date can be processed correctly. Then, new features such as month, semester, and day are extracted from the exam date to get more useful information from the dataset. After that, the total_marks column is normalized so that all values are brought into a similar range, which helps improve model performance. Finally, outliers in marks are detected and handled using IQR or Z-score methods to reduce the effect of unusual values. This preprocessing makes the dataset more accurate, clean, and suitable for student performance analysis and prediction.

Code:

```
df['month'] = df['exam_date'].dt.month
df['day'] = df['exam_date'].dt.day
df['semester'] = df['exam_date'].apply(lambda x: 1 if x.month <= 6 else 2)

print("DataFrame with new date features:")
display(df.head())
```

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df['total_marks_normalized'] = scaler.fit_transform(df[['total_marks']])

print("DataFrame with normalized 'total_marks':")
display(df.head())
```

```python
Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"Lower Bound (outlier threshold): {lower_bound}")
print(f"Upper Bound (outlier threshold): {upper_bound}")

# Identify outliers
outliers = df[(df['total_marks'] < lower_bound) | (df['total_marks'] > upper_bound)]

if not outliers.empty:
    print("\nDetected Outliers in 'total_marks':")
    display(outliers)

    # Handle outliers by capping them
    df['total_marks_cleaned'] = df['total_marks'].clip(lower=lower_bound, upper=upper_bound)
    print("\nDataFrame after capping outliers in 'total_marks':")
    display(df.head())
else:
    print("\nNo significant outliers detected in 'total_marks' based on IQR method.")
    df['total_marks_cleaned'] = df['total_marks']

print("\nFinal DataFrame after all preprocessing steps:")
display(df.head())
```

Output:

DataFrame with new date features:

	student_id	exam_date	total_marks	other_column	month	day	semester
0	1	2023-01-15	65	A	1	15	1
1	2	2023-02-20	88	B	2	20	1
2	3	2023-03-10	72	C	3	10	1
3	4	2023-04-05	95	D	4	5	1
4	5	2023-05-12	50	E	5	12	1

DataFrame with normalized 'total_marks':

	student_id	exam_date	total_marks	other_column	month	day	semester	total_marks_normalized
0	1	2023-01-15	65	A	1	15	1	0.333333
1	2	2023-02-20	88	B	2	20	1	0.844444
2	3	2023-03-10	72	C	3	10	1	0.488889
3	4	2023-04-05	95	D	4	5	1	1.000000
4	5	2023-05-12	50	E	5	12	1	0.000000

```
Q1: 66.25, Q3: 87.25, IQR: 21.0
Lower Bound (outlier threshold): 34.75
Upper Bound (outlier threshold): 118.75

No significant outliers detected in 'total_marks' based on IQR method.

Final DataFrame after all preprocessing steps:
```

	student_id	exam_date	total_marks	other_column	month	day	semester	total_marks_normalized	total_mark
0	1	2023-01-15	65	A	1	15	1	0.333333	
1	2	2023-02-20	88	B	2	20	1	0.844444	
2	3	2023-03-10	72	C	3	10	1	0.488889	
3	4	2023-04-05	95	D	4	5	1	1.000000	
4	5	2023-05-12	50	E	5	12	1	0.000000	

Explanation:

- **Convert exam_date into datetime format**
  This is done so that the date values can be read and processed correctly by the system.
- **Extract month from exam_date**
  The month helps in understanding in which part of the year the exam was conducted.
- **Extract semester from exam_date**
  The semester feature helps in grouping exams based on academic periods.
- **Extract day from exam_date**
  The day value can be useful for identifying exam patterns or trends.
- **Normalize total_marks**
  This step brings marks into a common range, which helps improve data consistency.
- **Detect outliers in marks**
  Outliers are unusual or extreme mark values that may affect analysis.

- **Handle outliers using IQR or Z-score**
  This helps in reducing the impact of abnormal values and makes the data more reliable.
- **Prepare dataset for analysis**
  After preprocessing, the dataset becomes clean and ready for prediction or performance analysis.

Task 3 – Student Health Data Preparation
Task:
Clean and preprocess student health records using AI scripts.
Instructions:
• Handle missing values in height, weight, BMI
• Encode categorical features (medical_condition, fitness_status)
• Scale numerical features using standard scaling
• Split dataset into training and testing sets

Prompt:

I have a **student health dataset** and I want to clean it and prepare it properly for analysis.

Fill the missing values in **height, weight, and BMI**

- Convert text columns like **medical_condition** and **fitness_status** into numbers
- Scale the numerical values using **standard scaling**
- Split the data into **training data** and **testing data**

I have a student health dataset and I want to clean it and prepare it properly for analysis. Fill the missing values in height, weight, and BMI • Convert text columns like medical_condition and fitness_status into numbers • Scale the numerical values using standard scaling • Split the data into training data and testing data

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# --- Step 1: Create Sample DataFrame and Initial Inspection ---

# Explicitly create the sample DataFrame for student health, overwriting any previous df
data = {
    'student_id': range(1, 16),
    'height': [160, 175, 163, np.nan, 180, 165, 170, 162, 178, 171, 166, 173, np.nan, 168, 176],
    'weight': [55, 70, 60, 72, 85, np.nan, 68, 58, 77, 65, 62, 71, 75, 63, np.nan],
    'BMI': [21.5, 22.8, 22.6, 23.5, 26.2, 24.0, np.nan, 22.1, 24.3, 22.2, 22.5, 23.7, 25.0, np.nan, 25.4],
    'medical_condition': ['None', 'Asthma', 'None', 'Diabetes', 'None', 'Asthma', 'None', 'None', 'Diabetes', 'None', 'Asthma', 'None', 'None', 'Diabetes', 'None'],
    'fitness_status': ['Fit', 'Average', 'Fit', 'Poor', 'Fit', 'Average', 'Fit', 'Poor', 'Average', 'Fit', 'Average', 'Fit', 'Poor', 'Fit', 'Average'],
    'health_status': [0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1] # Dummy target variable for splitting
}
df = pd.DataFrame(data)

print("Original DataFrame head:")
display(df.head())
print("\nDataFrame Info (before cleaning):")
df.info()

# --- Step 2: Fill Missing Values ---
for col in ['height', 'weight', 'BMI']:
    if col in df.columns:
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Missing values in '{col}' filled with median: {median_val}")

print("\nDataFrame head after filling missing values:")
display(df.head())
print("\nMissing values count after imputation:")
print(df.isnull().sum())

# --- Step 3: Convert Text Columns into Numerical Format ---
df = pd.get_dummies(df, columns=['medical_condition', 'fitness_status'], drop_first=True)

print("\nDataFrame head after one-hot encoding:")
display(df.head())

# --- Step 4: Scale Numerical Values ---
# Identify numerical columns to scale (exclude student_id and dummy variables and the target variable)
numerical_cols = ['height', 'weight', 'BMI']

scaler = StandardScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

print("\nDataFrame head after Standard Scaling numerical features:")
display(df.head())

# --- Step 5: Split Data into Training and Testing Sets ---
# Separate features (X) and target (y)
X = df.drop(['student_id', 'health_status'], axis=1) # Drop student_id as it's an identifier, and the target
y = df['health_status'] # Our dummy target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y) # Using stratify for balanced classes if target is categorical

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())

print("\nFirst 5 rows of y_train:")
display(y_train.head())
```

Output:

```
Original DataFrame head:
   student_id  height  weight  BMI  medical_condition  fitness_status  health_status
0           1   160.0    55.0  21.5               None             Fit              0
1           2   175.0    70.0  22.8             Asthma         Average              1
2           3   163.0    60.0  22.6               None             Fit              0
3           4     NaN    72.0  23.5           Diabetes            Poor              1
4           5   180.0    85.0  26.2               None             Fit              0

DataFrame Info (before cleaning):
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   student_id         15 non-null     int64
 1   height             13 non-null     float64
 2   weight             13 non-null     float64
 3   BMI                13 non-null     float64
 4   medical_condition  15 non-null     object
 5   fitness_status     15 non-null     object
 6   health_status      15 non-null     int64
dtypes: float64(3), int64(2), object(2)
memory usage: 972.0+ bytes
Missing values in 'height' filled with median: 170.0
Missing values in 'weight' filled with median: 68.0
Missing values in 'BMI' filled with median: 23.5

DataFrame head after filling missing values:
/tmp/ipykernel_1219/478616692.py:29: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(median_val, inplace=True)
   student_id  height  weight  BMI  medical_condition  fitness_status  health_status
0           1   160.0    55.0  21.5               None             Fit              0
1           2   175.0    70.0  22.8             Asthma         Average              1
2           3   163.0    60.0  22.6               None             Fit              0
3           4   170.0    72.0  23.5           Diabetes            Poor              1
4           5   180.0    85.0  26.2               None             Fit              0

Missing values count after imputation:
student_id          0
height              0
weight              0
BMI                 0
```

Explanation:

- **Handle missing values in height, weight, and BMI**
  Missing values are filled so that the dataset becomes complete and useful for analysis.
- **Encode medical_condition**
  This text column is converted into numbers so the system can understand it.
- **Encode fitness_status**
  This categorical feature is also changed into numerical form for model use.
- **Apply standard scaling**
  Numerical values are scaled to a common range so that all features are treated equally.
- **Split dataset into training and testing sets**
  The dataset is divided into two parts to train the model and test its performance.

- **Improve data quality**
  These preprocessing steps make the dataset clean, organized, and ready for prediction.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning
Scenario:
A ride-sharing company has trip data with errors.

Requirements:
• Standardize pickup and drop locations
• Fill missing fare amounts with median values
• Remove duplicate ride entries
• Normalize driver ratings
• Generate a summary of data improvements

Prompt:

I have a **ride-sharing trip dataset** that contains some errors and missing values.
Please help me **clean and preprocess** the data in a simple and understandable way.

Tasks to perform

- Standardize **pickup** and **drop locations**
- Fill missing **fare amounts** using the **median value**
- Remove **duplicate ride entries**
- Normalize **driver ratings**
- Generate a **summary of data improvements** after cleaning

I have a ride-sharing trip dataset that contains some errors and missing values. Please help me clean and preprocess the data in a simple and understandable way. Tasks to perform Standardize pickup and drop locations Fill missing fare amounts using the median value Remove duplicate ride entries Normalize driver ratings Generate a summary of data improvements after cleaning

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# --- Step 1: Create a sample Ride-Sharing Dataset ---

# Create a sample DataFrame for demonstration
data = {
    'trip_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    'pickup_location': [
        'Downtown', ' Airport', '  Downtown',
        'Suburbia', 'Airport ', 'Downtown',
        'Suburbia', 'City Center', 'Airport',
        'City center', 'Downtown', 'Airport'
    ],
    'dropoff_location': [
        'Uptown', '  Hotel', 'Uptown  ',
        'University', 'Office', 'Uptown',
        'University', '  Office', 'Hotel',
        'University', 'Office', 'Hotel'
    ],
    'fare_amount': [
        15.50, 25.00, 10.00,
        30.25, 27.50, np.nan,
        35.00, 18.75, 26.00,
        np.nan, 15.00, 27.50
    ],
    'driver_rating': [
        4.5, 3.8, 5.0,
        4.2, 4.5, 4.0,
        4.8, 3.9, 4.9,
        4.1, 4.7, 2.6
    ],
    'timestamp': pd.to_datetime([
        '2023-01-01 14:05:00', '2023-01-01 08:30:00', '2023-01-01 11:00:00',
        '2023-01-01 15:15:00', '2023-01-01 12:00:00', '2023-01-01 12:30:00',
        '2023-01-02 08:00:00', '2023-01-02 08:30:00', '2023-01-02 09:00:00',
        '2023-01-02 09:15:00', '2023-01-02 10:00:00', '2023-01-02 10:30:00'
    ])
}
df = pd.DataFrame(data)

# Introduce some explicit duplicates
df = pd.concat([df, df.iloc[[0, 5]]], ignore_index=True)

print("Original DataFrame head with introduced issues:")
display(df.head(7))
print("\nOriginal DataFrame info:")
df.info()
print("\nInitial missing values:")
print(df.isnull().sum())
print("\nInitial number of rows (including duplicates):", len(df))

# --- Step 2: Standardize Pickup and Drop Locations ---
df['pickup_location_cleaned'] = df['pickup_location'].str.strip().str.lower()
df['dropoff_location_cleaned'] = df['dropoff_location'].str.strip().str.lower()

print("\nDataFrame head after standardizing locations:")
display(df[['pickup_location', 'pickup_location_cleaned', 'dropoff_location', 'dropoff_location_cleaned']].head(7))

# --- Step 3: Fill Missing Fare Amounts using the Median ---
median_fare = df['fare_amount'].median()
df['fare_amount_cleaned'] = df['fare_amount'].fillna(median_fare)

print(f"\nMedian fare amount used for imputation: {median_fare:.2f}")
print("\nDataFrame head after filling missing fare amounts:")
display(df[['fare_amount', 'fare_amount_cleaned']].head(7))
print("\nMissing values in 'fare_amount_cleaned' after imputation:")
print(df['fare_amount_cleaned'].isnull().sum())

# --- Step 4: Remove Duplicate Ride Entries ---
initial_rows = len(df)
df_cleaned = df.drop_duplicates()
removed_duplicates = initial_rows - len(df_cleaned)

print(f"\nInitial number of rows: {initial_rows}")
print(f"Number of duplicate entries removed: {removed_duplicates}")
print(f"Number of rows after removing duplicates: {len(df_cleaned)}")
print("\nDataFrame head after removing duplicates:")
display(df_cleaned.head(7))

# --- Step 5: Normalize Driver Ratings ---
scaler = MinMaxScaler()
df_cleaned['driver_rating_normalized'] = scaler.fit_transform(df_cleaned[['driver_rating']])

print("\nOriginal 'driver_rating' vs Normalized 'driver_rating_normalized':")
display(df_cleaned[['driver_rating', 'driver_rating_normalized']].head(7))
print(f"Min normalized rating: {df_cleaned['driver_rating_normalized'].min():.3f}")
print(f"Max normalized rating: {df_cleaned['driver_rating_normalized'].max():.3f}")

# --- Step 6: Generate a Summary of Data Improvements ---
print("\n--- Data Cleaning and Preprocessing Summary ---")
print("1. **Location Standardization:** 'pickup_location' and 'dropoff_location' columns were converted to lowercase and leading/trailing spaces were removed. Example: '  Downtown' -> ' downtown'.")
print("2. **Missing Fare Amounts Filled:** {df['fare_amount'].isnull().sum()} missing values in 'fare_amount' were filled with the median value ({median_fare:.2f}).")
print("3. **Duplicate Entries Removed:** {removed_duplicates} duplicate ride entries were identified and removed, reducing the dataset from {initial_rows} to {len(df_cleaned)} rows.")
print("4. **Driver Ratings Normalized:** 'driver_rating' values were scaled to a 0-1 range. The original range was from {df_cleaned['driver_rating'].min():.1f} to {df_cleaned['driver_rating'].max():.1f}, and now the normalized range is from {df_cleaned['driver_rating_normalized'].min():.3f} to {df_cleaned['driver_rating_normalized'].max():.3f}.")

print("\nFinal Cleaned DataFrame head:")
display(df_cleaned.head())
print("\nFinal Cleaned DataFrame info:")
df_cleaned.info()
print("\nFinal missing values count:")
print(df_cleaned.isnull().sum())
```

Output:

```
5          4.8                0.90
8          4.0                0.50
Min normalized rating: 0.00
Max normalized rating: 1.00

--- Data Cleaning and Preprocessing Summary ---
1. **Location Standardization:** 'pickup_location' and 'dropoff_location' columns were converted to lowercase and leading/trailing spaces were removed. Example: '  Downtown' -> ' downtown'.
2. **Missing Fare Amounts Filled:** 3 missing values in 'fare_amount' were filled with the median value (24.50).
3. **Duplicate Entries Removed:** 2 duplicate ride entries were identified and removed, reducing the dataset from 14 to 12 rows.
4. **Driver Ratings Normalized:** 'driver_rating' values were scaled to a 0-1 range. The original range was from 3.0 to 5.0, and now the normalized range is from 0.00 to 1.00.

Final Cleaned DataFrame head:
```

	trip_id	pickup_location	dropoff_location	fare_amount	driver_rating	timestamp	pickup_location_cleaned	dropoff_location_cleaned	fare_amount_cleaned	driver_rating_normalized
0	1	Downtown	Uptown	15.50	4.5	2023-01-01 10:00:00	downtown	uptown	15.50	0.75
1	2	Airport	Hotel	25.00	3.8	2023-01-01 10:30:00	airport	hotel	25.00	0.40
2	3	Downtown	Uptown	16.00	5.0	2023-01-01 11:00:00	downtown	uptown	16.00	1.00
3	4	Suburbia	University	30.25	4.2	2023-01-01 11:15:00	suburbia	university	30.25	0.60
4	5	Airport	Office	24.50	3.5	2023-01-01 12:00:00	airport	office	24.50	0.25

```
Final Cleaned DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 12 entries, 0 to 11
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   trip_id                 12 non-null     int64
 1   pickup_location         12 non-null     object
 2   dropoff_location        12 non-null     object
 3   fare_amount             10 non-null     float64
 4   driver_rating           12 non-null     float64
 5   timestamp               12 non-null     datetime64[ns]
 6   pickup_location_cleaned 12 non-null     object
 7   dropoff_location_cleaned 12 non-null    object
 8   fare_amount_cleaned     12 non-null     float64
 9   driver_rating_normalized 12 non-null    float64
dtypes: datetime64[ns](1), float64(4), int64(1), object(4)
memory usage: 1.0+ KB

Final missing values count:
trip_id                    0
pickup_location            0
dropoff_location           0
fare_amount                2
driver_rating              0
timestamp                  0
pickup_location_cleaned    0
dropoff_location_cleaned   0
fare_amount_cleaned        0
driver_rating_normalized   0
dtype: int64
```

Explanation:

- **Standardize pickup and drop locations**
  Location names are made consistent to avoid spelling or format differences.
- **Fill missing fare amounts**
  Missing fare values are replaced using the **median value** to keep the data balanced.
- **Remove duplicate ride entries**
  Repeated trip records are deleted to avoid incorrect analysis.
- **Normalize driver ratings**
  Driver rating values are scaled into a common range for better comparison.
- **Generate summary of data improvements**
  A final summary is created to show what changes were made during cleaning.

- **Prepare data for analysis**
  After cleaning, the dataset becomes more accurate, organized, and useful.